

Assignment 1: AI Environment Build

SOC 910: How to Use AI for Sociological Research, Fall 2026

ChangHwan Kim (chkim@ku.edu) · 100 points

Due: Tuesday, September 29, before class

1 What You Are Turning In

You already built the environment in Weeks 2 through 5: VS Code with Claude Code, a git-tracked project folder, at least one MCP server, and the customization we covered in Week 4. Now you make it portable. You put the configuration in a GitHub repository so that it can be installed again somewhere else, and you send me the link. Because this is due on September 29, everything through the September 22 class counts, including the Stata and R connection.

Nothing new has to be installed. The work is collecting files you already have, removing any passwords or keys, and writing a short report.

2 How to Submit

1. Create a GitHub repository named `soc910-env-lastname`, for example `soc910-env-dobbs`.
2. Private is fine. If it is private, add me as a collaborator: GitHub user `kchyhj`, under Settings → Collaborators.
3. Push the six items in Section 3.
4. Email me the repository link by Tuesday, September 29, before class.

3 What Goes in the Repository

The repository is not a pile of evidence. It is your environment in portable form: you clone it onto your office desktop, your laptop, or your next computer, follow your own instructions, and end up with the same setup. Write it for yourself, six months from now, on a machine with nothing installed.

Six things belong in it. The folder names are a suggestion, not a rule, as long as your `README.md` says where each one is. If an item does not apply to you, say so in one line rather than leaving it out silently.

#	Path	What goes there
1	<code>README.md</code>	What this environment is, how to install it on a new machine, and your build report. See Section 4.
2	<code>setup/</code>	What has to exist before anything else works: <code>versions.txt</code> with the version of Claude Code, Git, Python, Node.js, and VS Code, and <code>install.md</code> with the steps in order, starting with your operating system.
3	<code>claude/</code>	The configuration itself, ready to be copied into <code>~/.claude</code> and into a project folder: your user-level and project <code>CLAUDE.md</code> , <code>settings.json</code> , and at least one skill, slash command, or hook you wrote.
4	<code>mcp/</code>	Your MCP server configuration with keys replaced by placeholders, and the one-line command that registers it. The Stata server from Week 5 counts.
5	<code>project/</code>	How you organize research folders: the <code>.gitignore</code> you reuse, your folder structure, and the last 20 commits as a sample of real use.
6	<code>screenshots/</code>	Four images: VS Code with Claude Code open, the <code>/mcp</code> output showing your server connected, the agent finishing a real task, and your commit history.

4 The Build Report (`README.md`)

Three to five pages with your screenshots in it. Answer these seven questions.

1. What machine is this, and was anything unusual about it? A university-managed laptop, limited permissions, a non-English system.
2. What did you install, and what is each piece for?
3. What broke, what did the error actually say, and how did you fix it? If nothing broke, say so.
4. Which MCP server did you choose, why does it fit your research, and what did you ask the agent that it could answer only through that server?
5. How is your project folder organized, what does your `CLAUDE.md` tell the agent, and what did you keep out of git?
6. When you clone this repository onto another of your machines and follow your `install.md`, what do you get, and what still has to be done by hand? Accounts, licenses, and API keys do not travel, so list them.
7. What did you customize, and what problem does it solve? Name the skill, slash command, or hook, and say what you were tired of repeating.

Question 3 is the one I read most closely. It is where a real build shows.

5 Let Claude Package the Environment

Packaging a setup so it can be installed again is exactly the kind of work the agent should do. Open Claude Code in your research project folder and give it something like this.

Copy everything inside this box

I want to package my Claude Code environment so I can reinstall it on another computer. Build the repository in ~/soc910-env-<lastname>.

1. `claude/`: copy my user CLAUDE.md, this project's CLAUDE.md, settings.json, and the skills, commands, and hooks I wrote, in their current layout.
2. `mcp/`: the configuration for my connected MCP servers, plus the command that registers each one on a new machine.
3. `project/`: my .gitignore, this folder's structure two levels deep, and the last 20 commits.
4. `setup/versions.txt`: versions of Claude Code, git, python, node, and VS Code.
5. `setup/install.md`: ordered steps to rebuild this on a computer with nothing installed, ending with how to check that it worked.
6. Replace every key, token, and password with a placeholder, then show me each file you changed.
7. List in `install.md` what cannot be reproduced: accounts, licenses, paid subscriptions, machine-specific paths.
8. `README.md`: what this environment is, a short install summary, and the seven report questions as empty headings.
9. `git init`, one commit, and the commands to push to a new GitHub repository.

Ask me before copying anything that might contain participant data.

Three rules when you do this.

1. **Check what it produced.** Open every file before you push. You are responsible for anything in the repository, including a key the agent missed.
2. **Write the report yourself.** The agent can set up the headings. The answers, especially questions 3 and 7, have to be yours; they are what I am grading.
3. **Test the instructions.** Read `setup/install.md` as if you had a new laptop and nothing installed. Every step you could not actually follow is a step to fix before you push.

6 Never Commit These

- API keys, tokens, passwords, .env files. Replace the value with REDACTED and keep the rest of the line so the setting still makes sense.
- Participant data, restricted-use data, anything under an IRB protocol. Show the folder structure, not the contents.
- Files over about 10 MB, and other people's unpublished work.

If you find that a key was committed earlier in your project's history, tell me instead of quietly rewriting the history. Changing the key matters more than cleaning up the record.

7 Checklist

- ☐ Claude Code runs inside VS Code and finishes a multi-step task on your machine.

- ☐ Your research folder is a git repository with commits of your own work.
- ☐ One MCP server is connected and you have used it for something real.
- ☐ You wrote at least one skill, slash command, or hook, and it is in the repository.
- ☐ All six items from Section 3 are in the repository.
- ☐ You could clone this repository onto another of your own machines, follow `setup/install.md`, and rebuild your setup there.
- ☐ You opened every file and there are no keys or restricted data.
- ☐ The repository is shared with `kchyhj` and you have sent the link.

8 How It Is Graded

Four things have to be true.

1. Claude Code works inside VS Code.
2. Your research folder is tracked by git.
3. At least one MCP server is connected and useful to your research.
4. You wrote at least one skill, slash command, or hook.

Keep the customization small. A slash command that converts new PDFs, a hook that sweeps Stata log files, a skill that formats your summaries the same way every time: any of these is enough, as long as it solves a problem you actually had.

The report is where you show that the environment is yours rather than copied. Questions 3 and 7 matter most, because a failure you worked through and a rule you got tired of repeating are both hard to borrow from someone else.